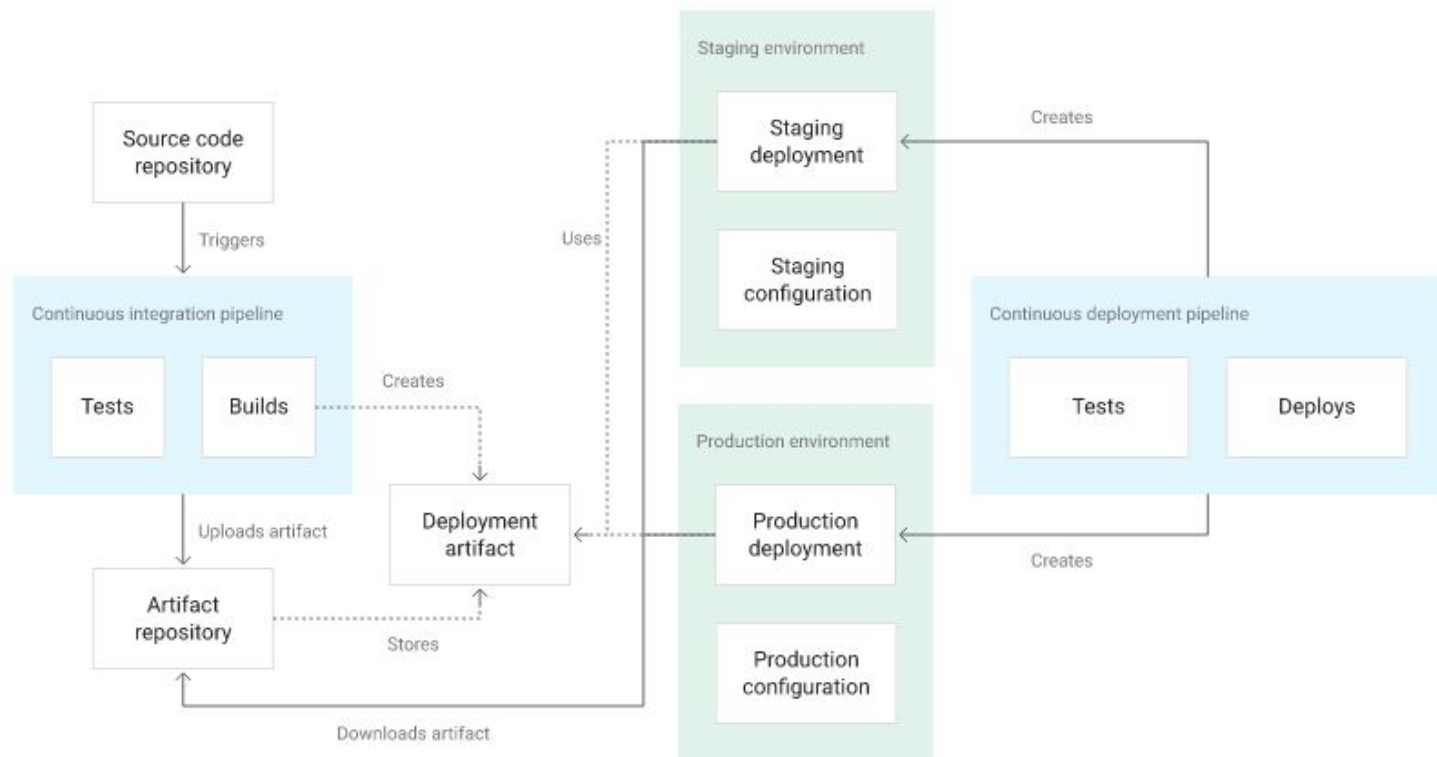

Web Application Deployment

Software Deployment

Deployment is a stage in software management where the **software is installed** on workstations or **hosted** as an application and accessible to others in a **runtime state**

Software or application **deployment is the process required to make new or updated software available to its users**

Software Deployment Components



Software Deployment Components

Deployment artifact

A deployment artifact, or **artifact**, is a packaged application or module that is ready to be deployed and installed

An artifact is immutable: it cannot be modified after it's created

Many different formats for artifacts exist, for example, JAR, WAR, gems, and container image

Artifacts are stored in an artifact storage system

Software Deployment Components

Deployment

The action of installing and configuring an artifact in a specific environment

The end result of the deployment action

Software Deployment Components

Release

A release is a specific artifact you deem stable enough to use for production

To create a release, you usually create several artifacts, each new one fixing issues from, or adding features to, the previous one until you reach the stability or feature capabilities you want

Those artifacts can have names such as **alpha**, **beta**, or **release candidate**

Software Deployment Components

Pipeline

A pipeline is a computing pattern that takes something as input, runs a series of processing jobs, and returns an output

You can serialize or parallelize these stages

Software Deployment Components

Continuous integration

Continuous integration (CI) is a methodology in which developers test and build (integrate) their code changes as often as possible

The goal of CI is to tighten the development feedback loop and to surface errors and problems as early as possible in the development process

Software Deployment Components

Continuous integration

CI operates on the rationale that the later an error is discovered, the more expensive it is to fix

Discovering a defect in production is, of course, something most developers want to avoid

Software Deployment Components

Continuous integration

CI is usually implemented with pipelines that run periodically or are triggered when a developer pushes a code change

For example, a CI pipeline could have three jobs: one that runs a linter, one that runs unit tests, and one that builds an artifact from the source code.

The usual output of a CI pipeline is an artifact in an artifact storage system

Software Deployment Components

Continuous delivery

Continuous delivery (CD) is the capability of releasing code at any time

CD assumes that your code has passed the CI pipeline and any tests that you deem necessary (such as smoke testing, QA, and load testing)

Software Deployment Components

Environment

An environment is the infrastructure or set of computing, networking, and storage resources on which you deploy your application

A set of configurations is usually associated with the environment for the applications it's hosting

These configurations usually vary from one environment to another

Software Deployment Components

Configuration

A configuration is a piece of information your application needs in order to run that isn't part of the deployment artifact

You can change your configuration without creating or deploying a new artifact

A configuration isn't specific to a single deployment artifact

Different configurations can have different properties

Software Deployment Components

Configuration

A configuration can have a value that varies by environment and is used by multiple applications in an environment

A configuration can have a value that varies by environment and is used by a single application

Typically, a configuration name stays the same across environments, while a configuration value varies across environments

Software Deployment Components

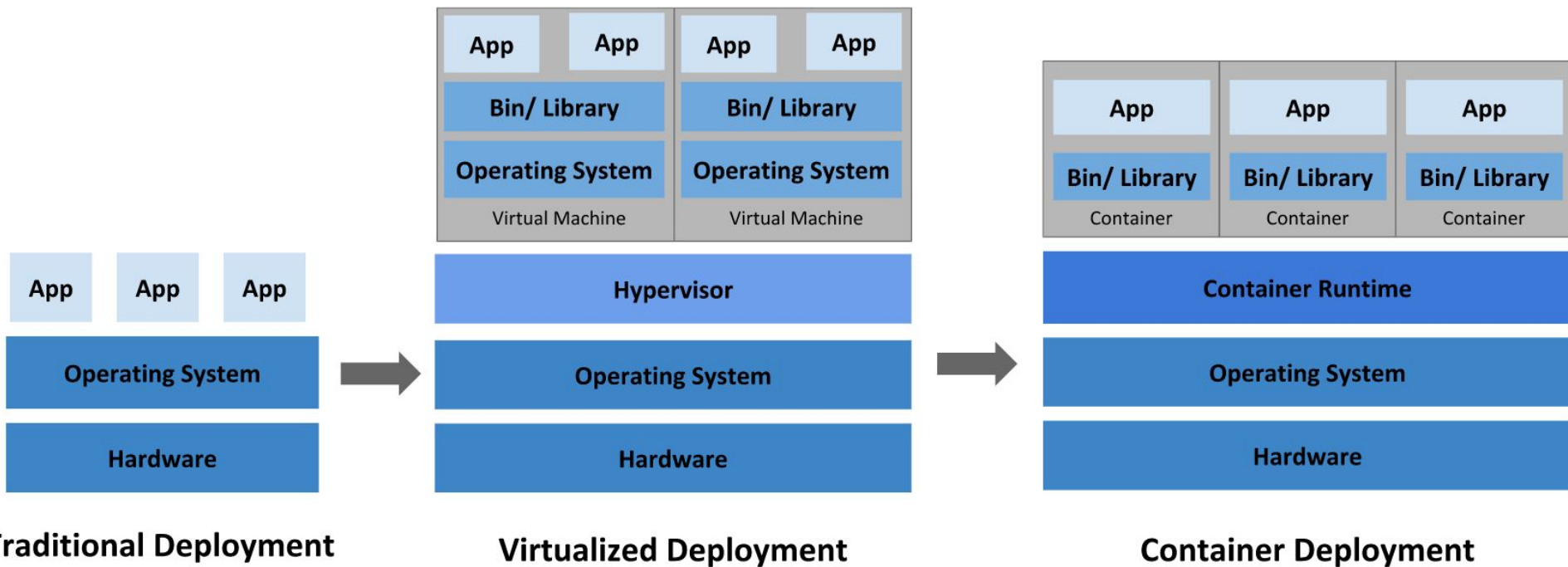
Secret

Secrets are a specific kind of configuration

Examples of secrets include database passwords, API keys for third-party services, and cryptographic keys

You should protect secrets to help prevent attackers from accessing what are otherwise private systems

Deployment Types

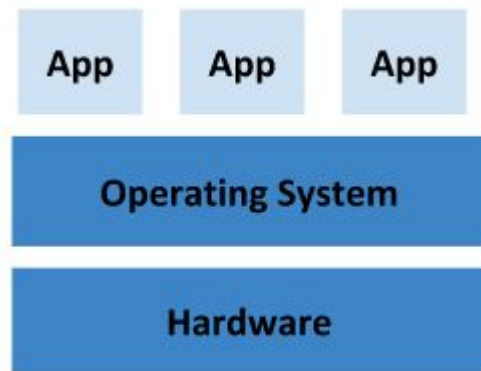


Deployment Types

Traditional deployment

run applications on physical servers

have resource allocation issues



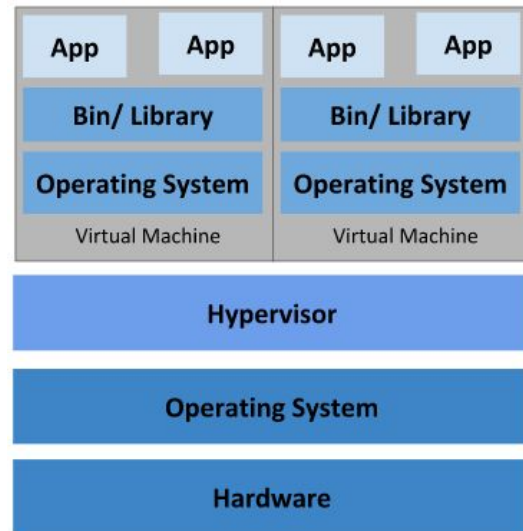
Traditional Deployment

Deployment Types

Virtualized deployment

allows you to run multiple Virtual Machines (VMs) on a single physical server CPU

allows applications to be isolated between VMs and provides a level of security as the information of one application cannot be freely accessed by another application

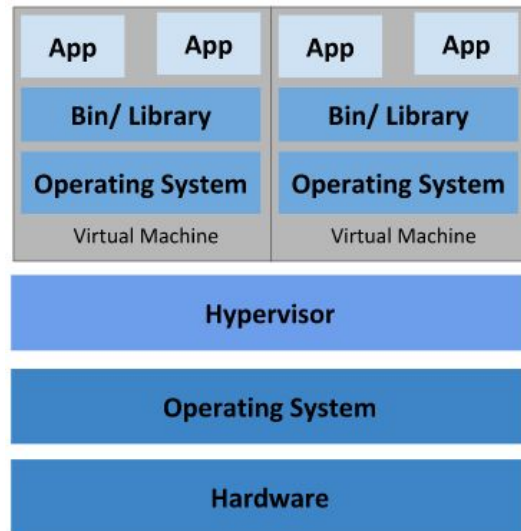


Virtualized Deployment

Deployment Types

Virtualized deployment

allows better utilization of resources in a physical server and allows better scalability because an application can be added or updated easily, reduces hardware costs, and much more



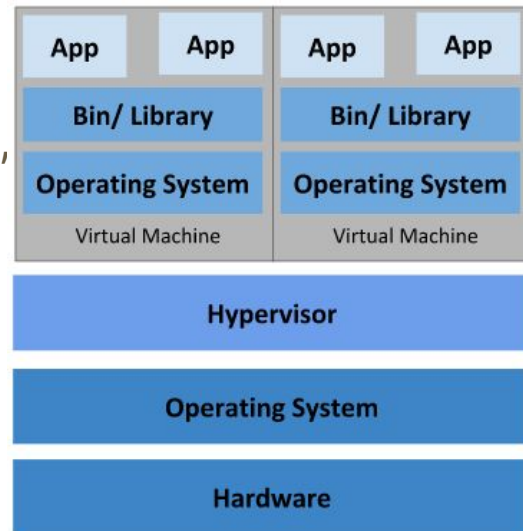
Virtualized Deployment

Deployment Types

Virtualized deployment

With virtualization you can present a set of physical resources as a cluster of disposable virtual machines

Each VM is a full machine running all the components, including its own operating system, on top of the virtualized hardware



Virtualized Deployment

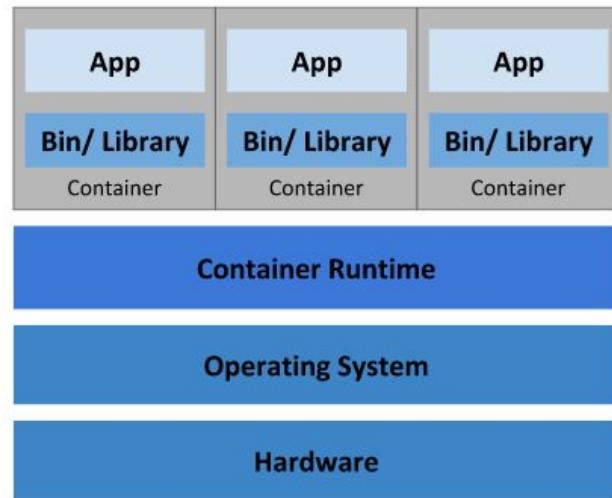
Deployment Types

Container deployment

share the Operating System (OS) among the applications, therefore, containers are considered lightweight

Similar to a VM, a container has its own filesystem, share of CPU, memory, process space, and more

As they are decoupled from the underlying infrastructure, they are portable across clouds and OS distributions



Container Deployment

Application Deployment Process

Plan

This stage is what gets development and operations teams on the same page

This is where the deployment schedule is mapped out, the current infrastructure is assessed, and changes are made if necessary

Application Deployment Process

Build and Release Automation

Enabling automation is key to a successful app deployment process, as the possibility for human error needs to be decreased as much as possible

Application Deployment Process

Develop Continuous Integration / Continuous Delivery (CI/CD)

Working to reduce the degree of change in each application update will help teams spot breakdowns in the future

When deployments are minimally impactful, your system will be able to carry out more frequent ones easier.

Application Deployment Process

Create and Test Scripts

Identify environmental changes and differences by running test scripts on a backup copy of production before moving to the final release

Application Deployment Process

Identify Key Metrics

Make sure your team is clear on key performance indicators (KPIs) from one application to another

Make sure KPIs are set, visibility is enabled, and any potential errors in the application set are troubleshoot swiftly

Application Deployment Process

Test

Set up synthetic transaction tests, and make sure key items like login pages are working correctly. Go into your deployment with confidence

Application Deployment Process

Develop Deployment Tracking

Enable and implement tracking services to make sure ops teams can easily track when deployments occur (or when they're scheduled to), and to immediately identify when errors occur and how to fix them

Application Deployment Process

Alert Users and Colleagues

Alert necessary constituents as to when an application is expected to deploy

This will help coordinate throughout the process, set expectations, and backtrack if there were any errors

Application Deployment Process

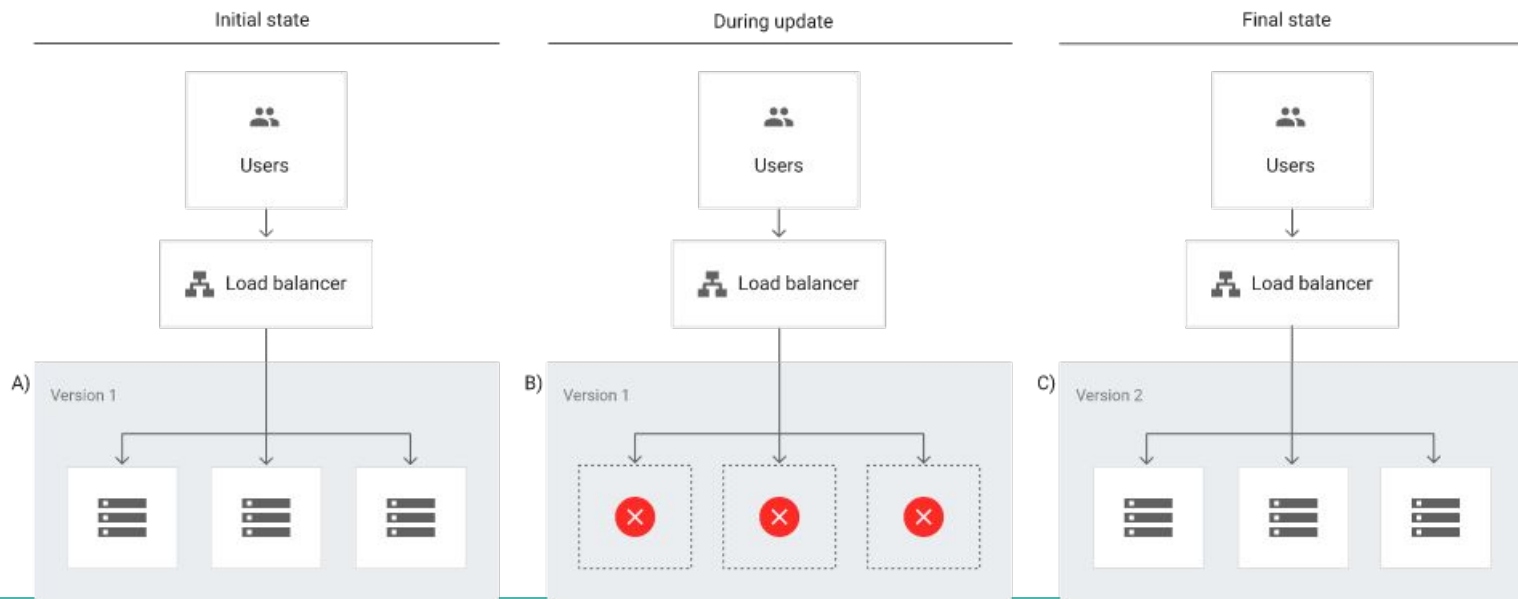
Monitor and Iterate

Once the application is deployed, it will be as equally as important to monitor deployment and correct as needed

Application Deployment Strategies

Recreate

Version 1 is terminated, and Version 2 is then deployed



Application Deployment Strategies

Ramped/Rolling

Version 2 is slowly rolled out and it replaces Version 1



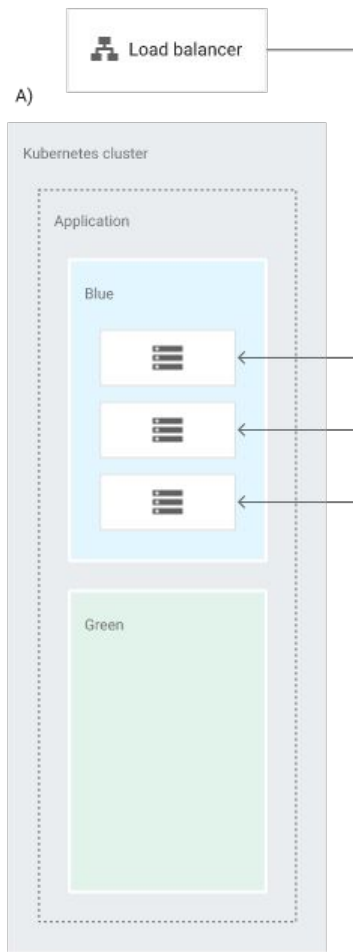
Application Deployment Strategies

Blue/Green Deployment

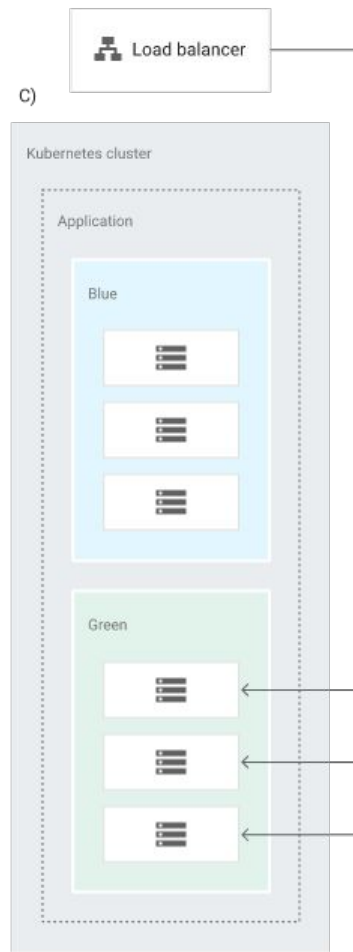
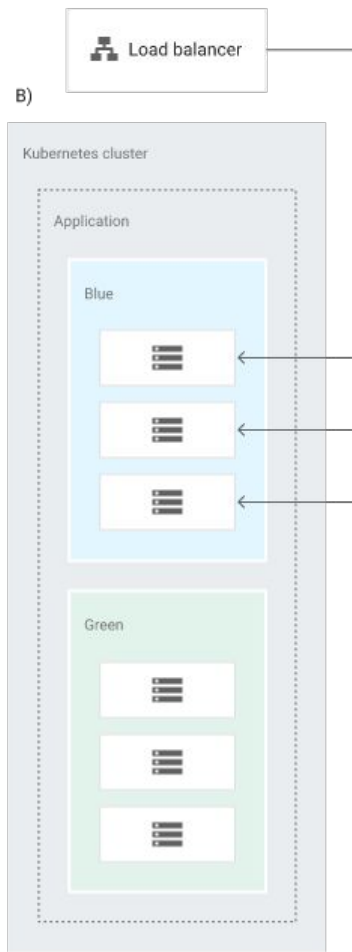
Version 2 is released alongside Version 1, and then the traffic is switched over to Version 2

Check the diagram in the next slide

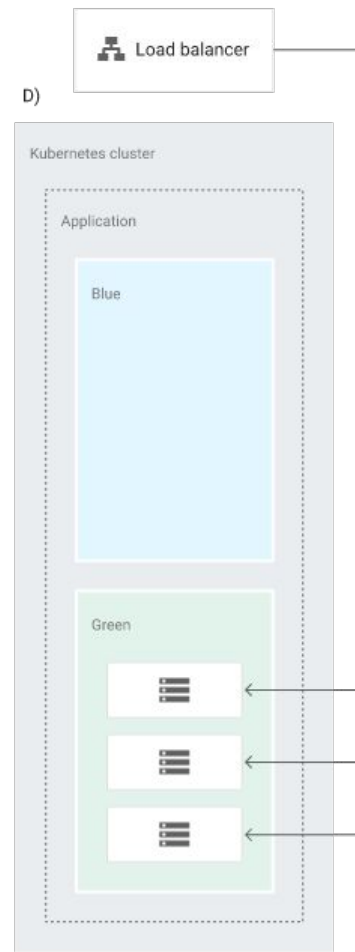
Initial state



During update



Final state

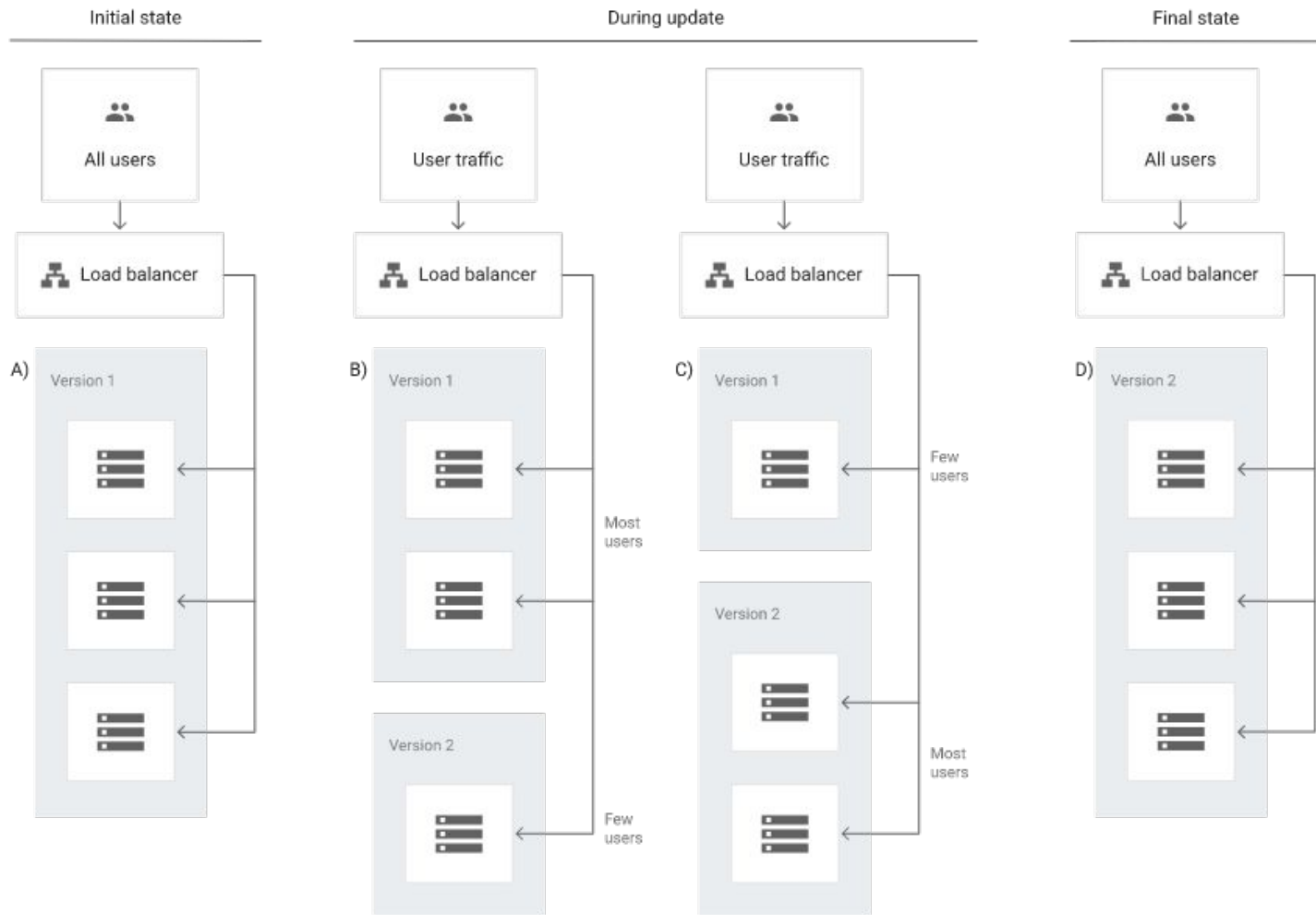


Application Deployment Strategies

Canary

Version 2 is released to a subgroup of users, then proceeds to a full rollout

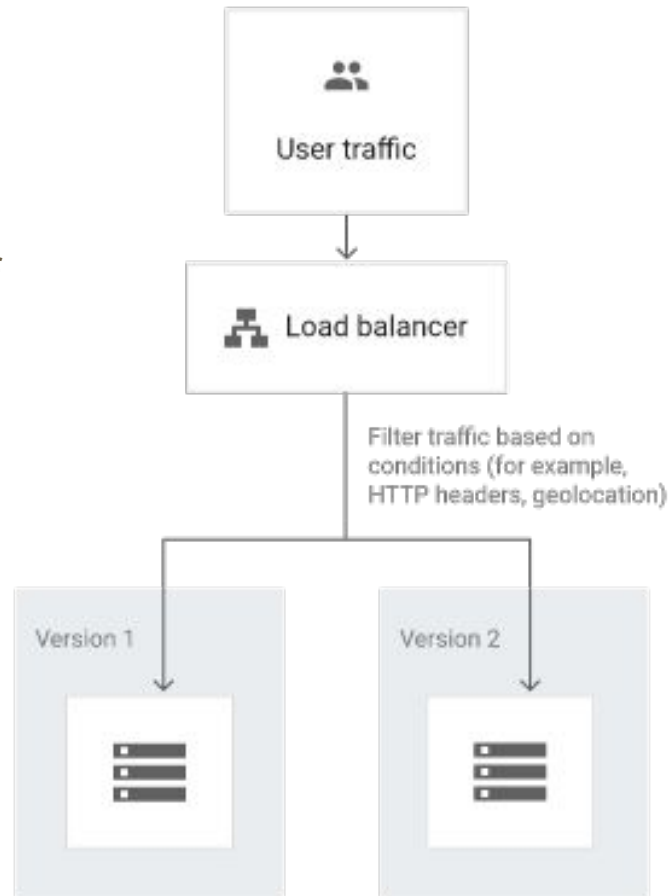
Check the diagram shown in the next slide



Application Deployment Strategies

A/B Testing

Version 2 is released to a particular subset of users under specific conditions

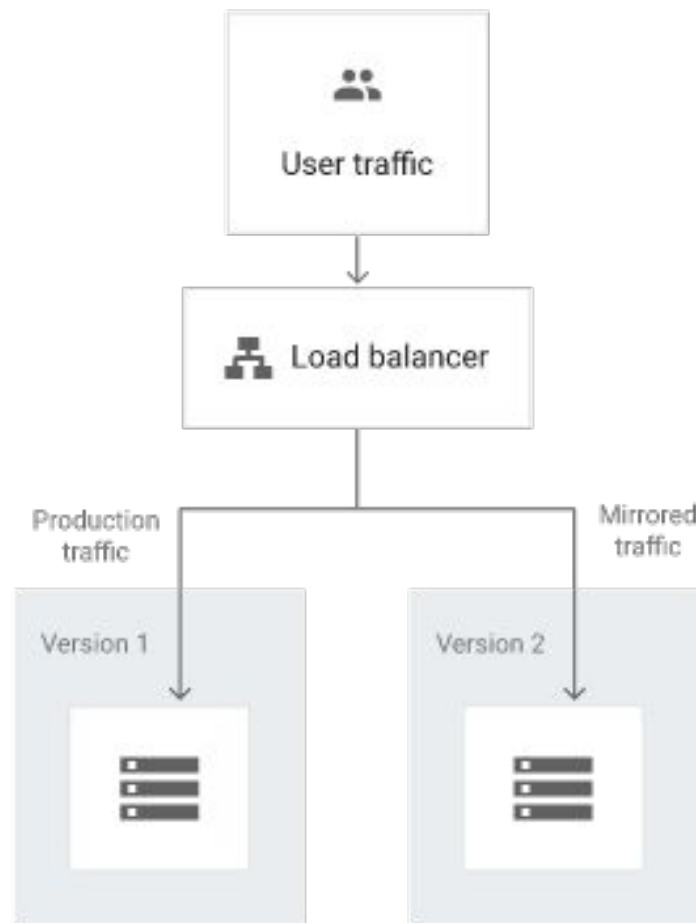


Application Deployment Strategies

Shadow

Version 2 receives real-world traffic in conjunction with Version 1

The traffic sent to Version 2 doesn't impact the response



Deployment Strategies Comparison

Deployment strategy	Zero downtime	Real production traffic testing	Releasing to users based on conditions	Rollback duration	Impact on hardware and cloud costs
Recreate	x	x	x	Fast but disruptive because of downtime	No extra setup required
Rolling update	✓	x	x	Slow	Can require extra setup for surge upgrades

Deployment Strategies Comparison

Deployment strategy	Zero downtime	Real production traffic testing	Releasing to users based on conditions	Rollback duration	Impact on hardware and cloud costs
Blue/green	✓	x	x	Instant	Need to maintain blue and green environments simultaneously
Canary	✓	✓	x	Fast	No extra setup required

Deployment Strategies Comparison

Deployment strategy	Zero downtime	Real production traffic testing	Releasing to users based on conditions	Rollback duration	Impact on hardware and cloud costs
A/B	✓	✓	✓	Fast	No extra setup required
Shadow	✓	✓	x	Does not apply	Need to maintain parallel environments in order to capture and replay user requests

References

<https://kubernetes.io/docs/concepts/overview/>

<https://cloud.google.com/architecture/addressing-continuous-delivery-challenges-in-a-kubernetes-world>

<https://cloud.google.com/architecture/application-deployment-and-testing-strategies>